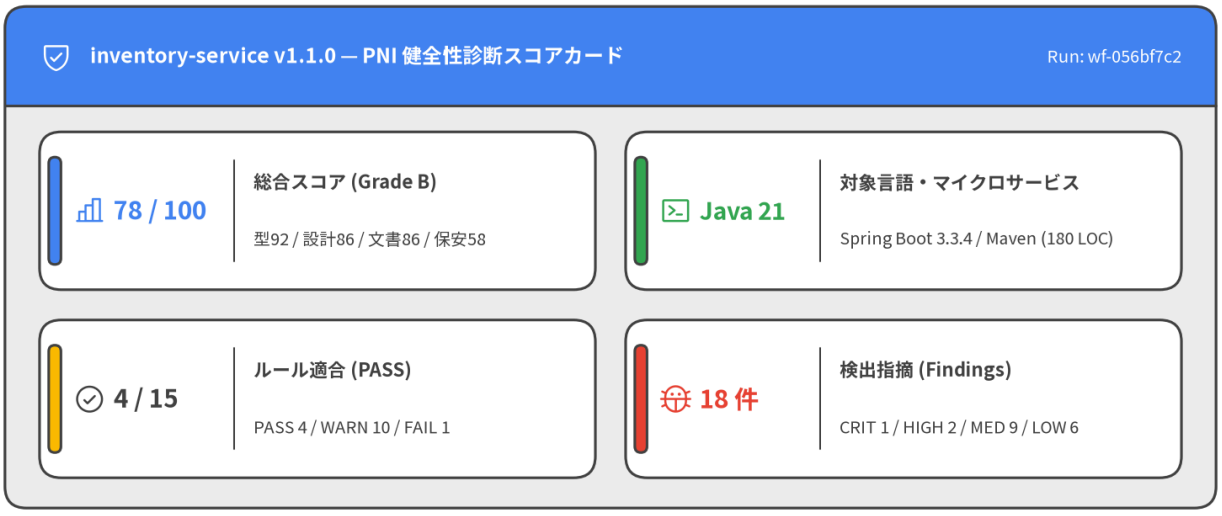
PNI ソフトウェア健全性診断レポート

本レポートは、倉庫在庫引当およびサプライヤー自動補充通知を担う Spring Boot 3.3 + Java 21 マイクロサービス `inventory-service` (`v1.1.0`) に対して、PNI ステートフル監査ワークフロー `java_api_spring_clean` （ビルド・供給網検査、Java AST 静的解析、3側面アーキテクチャ記録、隔離コンテナでのマルチステージビルド・オフライン JUnit 5 / JaCoCo カバレッジ計測・起動スモーク検証、Trivy 脆弱性・IaC・秘密情報スキャン、および `.pni/pentests/` による動的 Web ペネトレーションテスト `web_pentest` ）を実行した包括的診断結果である。



inventory-service (v1.1.0) PNI 総合健全性スコアカード

診断メタデータ概要

- **対象プロジェクト**: `inventory-service` (`v1.1.0` / 対象ルート: `examples/java/api_spring_clean`)
- **対象言語・ビルド基盤**: `java` (`21` , Apache Maven `pom.xml` , Spring Boot `3.3.4` Web + Validation, JUnit Jupiter `5.10.3`)
- **診断ワークフロー / Run ID**: `java_api_spring_clean` / `wf-056bf7c2`
- **評価実施日時**: `2026-10-24T14:47:03.897244+00:00`
- **総合スコア・グレード**: **78 / 100 (Grade B)**
- **エンジニアリングルール適合率**: `4 / 15` ルール PASS（ `WARN` : 10件、 `FAIL` : 1件）

- **動的ペネトレーション検証 (`web_pentest`)**: `passed` （`16` プローブ実行：`12` PASS / `3` FAIL / `1` SKIP、CRITICAL・HIGH 脆弱性 `0` 件）

Table of Contents

1. エグゼクティブサマリー

2. アーキテクチャとソースコード構成

3. 推奨する変更対応

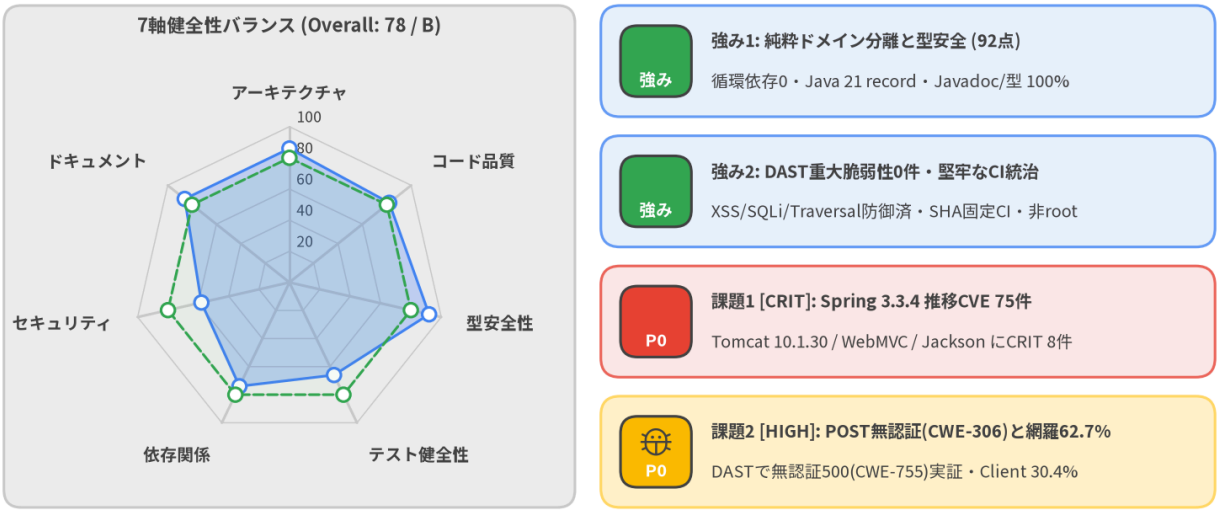
4. 検査指標・ルール適合マトリクスと指摘一覧

5. 詳細分析：コード品質・テスト・動的検証

6. 詳細分析：セキュリティ・脆弱性・API境界

1. エグゼクティブサマリー

`inventory-service` (`v1.1.0`) の総合健全性スコアは **78 / 100 (Grade B)** と評価された。Java 21 `record` とコンストラクタ DI に基づく 5パッケージ・180 LOC の明快な非循環レイヤード設計、ならびに Javadoc・ジェネリクス型安全・import 衛生すべて `100.0%` という高い静的品質を備え、マルチステージ Docker ビルド・オフライン JUnit 5 単体テスト・Spring Boot 起動検証、および隔離ループバック環境に対する動的 Web ペネトレーションテスト（`web_pentest`: 全 `16` プローブ中 `12` PASS・CRITICAL/HIGH 脆弱性 `0` 件）に合格している。一方で、`spring-boot-starter-parent:3.3.4` が引き込む推移依存の既知脆弱性（CRITICAL `8` 件・HIGH `29` 件）、ライブ DAST プローブでも実証された状態変更 API `POST /api/v1/items/{sku}/reserve` の無認証実行（`CWE-306`）および未知 SKU 時の未処理 `HTTP 500`（`CWE-755`）、ならびに JaCoCo 実測行網羅率 `62.7%`（分岐 `50.0%`）が最優先の解消課題である。



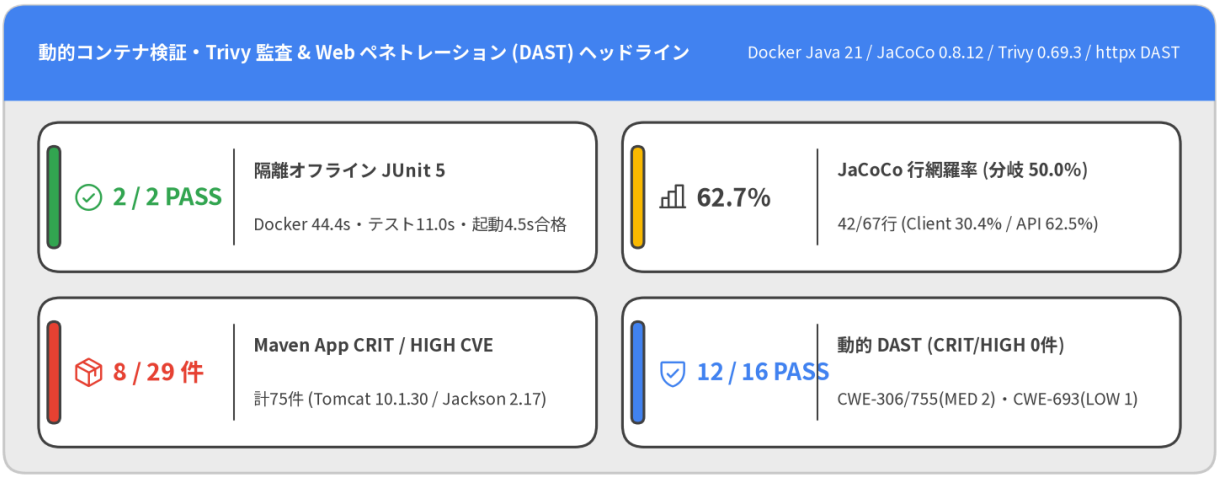
7軸エンジニアリング健全性レーダーと強み・最優先リスク要約

7軸評価サマリーと主要診断結論

- **高評価領域（型安全性 `92` ・アーキテクチャ `86` ・ドキュメント `86` ・コード品質 `82` ）**: `api` -> `service` -> `domain` + `client` + `config` の単方向 DAG が徹底され、`StockItem` ・ `ReorderPolicy` のドメイン層は Spring フレームワーク依存を一切持たない。GitHub Actions の 40文字 SHA ピン留め（`contents: read`）、Dependabot 週次更

新、`SECURITY.md`、ハードコード秘密情報ゼロ、危険な CWE コードシンクゼロに加え、`.pni/pentests/` によるライブ HTTP 侵入プローブでも不正数値の `400` 拒否（`CWE-20`）と XSS / SQLi / トラバーサル耐性が実証された。

- **優先改善領域（セキュリティ `58`・テスト健全性 `66`・依存関係 `74`）**: 全8監査ステップでは `security_vulnerability_audit`（`56 / D`）、`java_test_audit`（`68 / C`）、`web_penetration_audit`（`72 / B`）、`container_runtime_audit`（`76 / B`）が相対的な弱点となった。`pom.xml` の Spring Boot `3.3.4`（Tomcat `10.1.30`、Spring WebMVC `6.1.13`、Jackson `2.17.2`）更新、予約 `POST` エンドポイントへの OAuth2 JWT 認可導入と未知 SKU 例外の `404 ProblemDetail` 変換（`CWE-306` / `CWE-755`）、`X-Content-Type-Options: nosniff` ヘッダ付与（`CWE-693`）、`ConcurrentHashMap` のアトミック更新化、および HTTP クライアント・異常系のテスト拡充が急務である。



コンテナ動的検証・Trivyスキャン・Webペネトレーション検証の主要指標

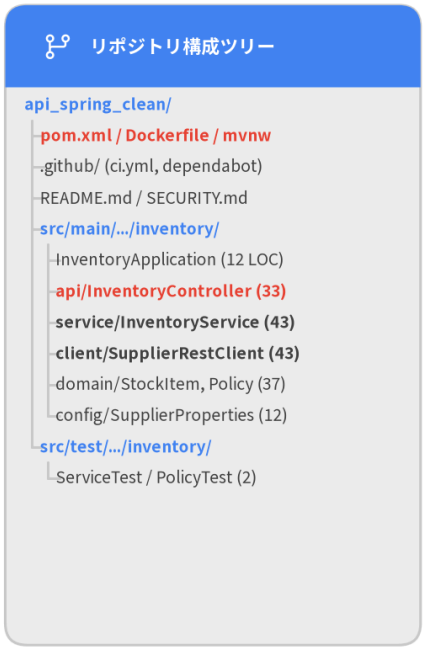
エグゼクティブ結論

- **強み (Verified Strengths)**: コンストラクタ DI と Java 21 `record` による高い凝集度、明示的な HTTP 接続・読取タイムアウト（`SupplierRestClient`）、非 root コンテナ実行（`USER spring`, `UID 10001`）とループバックバインド（`127.0.0.1`）、ライブ DAST で確認された型変換バリデーション（`quantity=not-an-int -> 400`）とインジェクション耐性、および Diátaxis 4象限に沿った簡潔なドキュメント構成が確認された。
- **優先対応 (Top Actionable Priorities)**: **P0** で Spring Boot BOM / Tomcat / Jackson のパッチ版更新と `POST /api/v1/items/{sku}/reserve` の OAuth2 JWT スコープ保護を実施し、**P1** で `@RestControllerAdvice`（`ProblemDetail` による未知 SKU `404` / 在庫不足

`409` 変換)・Actuator `/actuator/health` 導入、`SupplierRestClient` の Jackson シリアライズ化と `InventoryService` の `compute(...)` アトミック化、および JaCoCo 網羅率 `85%` 以上へのテスト拡充、さらに **P2** で `X-Content-Type-Options: nosniff` ヘッダ付与と供給網・文書整備を行うことで、本番外部公開に耐える Grade A 水準へ到達できる。

2. アーキテクチャとソースコード構成

`inventory-service` は **Layered Spring Boot Microservice Architecture（Bootstrap 層 -> API プレゼンテーション層 -> アプリケーションサービス層 -> 純粋ドメイン層 + 外部 HTTP クライアントアダプタ層 + 構成層）** を採用しており、全7ソースファイル（計 `180` LOC・7型・16メソッド/コンストラクタ）が単方向の非循環グラフを形成している。



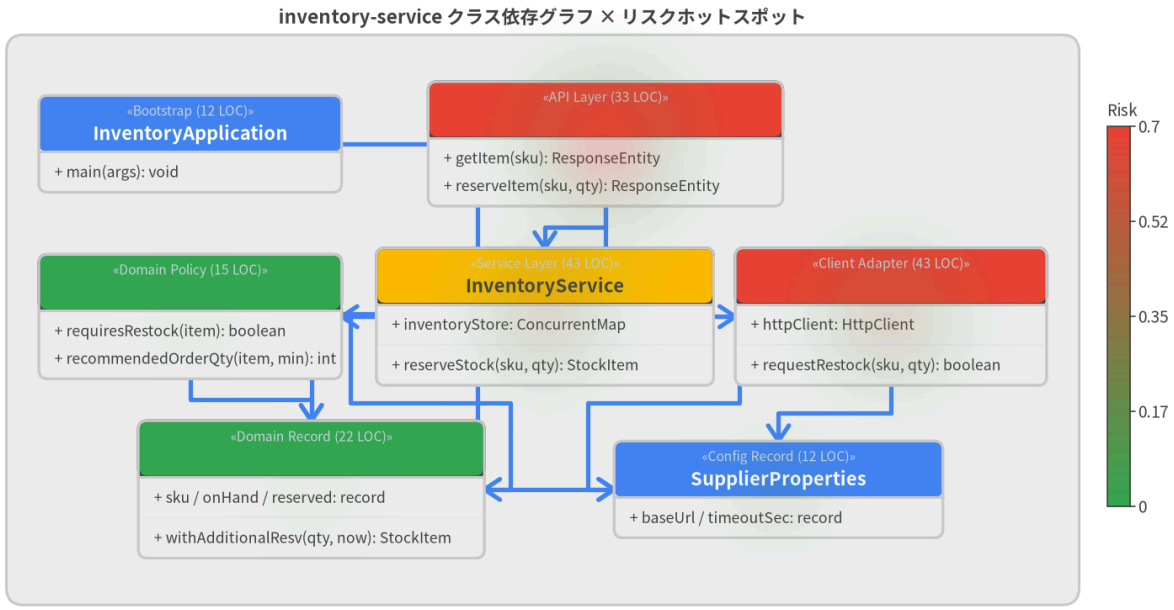
モジュール (相対パス要約)	LOC	公開型・メソッドと主要責務
InventoryApplication.java	12	InventoryApplication.main Spring起動・SupplierProperties有効化
api/ InventoryController.java	33	getItem, reserveItem GET /{sku}・POST /{sku}/reserve 公開
service/ InventoryService.java	43	upsertItem, findBySku, reserveStock 在庫引当更新と補充判定オーケストレーション
client/ SupplierRestClient.java	43	requestRestock (HttpClient) タイムアウト付 POST /v1/restock 送信
domain/ StockItem.java	22	availableUnits, withAdditionalReservation Java 21 record による不変在庫スナップショット
domain/ ReorderPolicy.java	15	requiresRestock, recommendedOrderQuantity 発注点判定および推奨補充数量の純粋計算
config/ SupplierProperties.java	12	SupplierProperties, defaults inventory.supplier.* 外部設定バインド

リポジトリディレクトリ構造ツリーとASTモジュールインベントリ (全7ファイル・180 LOC)

Spring レイヤー依存グラフとモジュールリスク密度（`HeatmapOverlay`）

AST 内部 import グラフ解析により、循環依存（`circular_dependencies = 0`）および上位層への逆依存（`boundary_violations = 0`）はゼロであることが検証された。`StockItem` と `ReorderPolicy` は外部ライブラリへの依存を持たない純粋ドメインモデルとして隔離されている。一方、指摘密度に基づくホットスポット（`module_hotspots`）では、無認証・例外変換・セキュリティヘッダ指摘（DAST 含む）が重なる `InventoryController.java`（`risk_score=0.64`）を筆頭に、推移 CVE を抱える `pom.xml`（`0.45`）、JSON 文字列連結と低カバレッジが重なる `SupplierRestClient.java`（`0.40`）、および未知 SKU `HTTP 500`

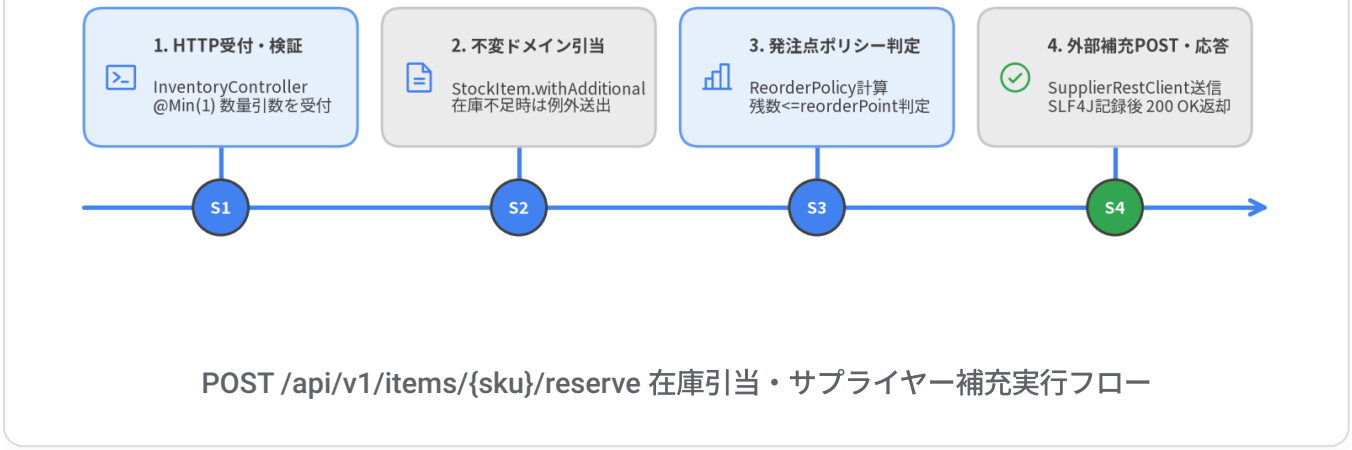
（`CWE-755`）・競合更新課題を持つ `InventoryService.java`（`0.36`）にリスクが集中している。



Spring Boot レイヤー依存関係グラフとモジュールホットスポット (HeatmapOverlay)

リクエスト実行フローと状態・並行制御モデル

本サービスは `GET /api/v1/items/{sku}`（在庫照会フロー）と `POST /api/v1/items/{sku}/reserve?quantity={n}`（在庫引当・条件付きサプライヤー補充フロー）の2系統の HTTP エントリポイントを持つ。状態は `InventoryService` 内のインメモリ `ConcurrentHashMap<String, StockItem>` に保持され、外部サプライヤー通信では `IOException` や `InterruptedException`（割り込みフラグ復元付き）を捕捉して `false` へグレースフルに縮退する設計となっている。



POST /api/v1/items/{sku}/reserve 在庫引当・サプライヤー補充実行フロー

3. 推奨する変更対応

検出された全18件の指摘事項（`CRITICAL: 1`、`HIGH: 2`、`MEDIUM: 9`、`LOW: 6`）を体系的に解消し、総合スコアを Grade B (`78`) から Grade A/S 水準（`90+`）へ引き上げるため、緊急度とリスク低減効果に基づき **P0（最優先セキュリティ遮断：2施策） -> P1（本番信頼性・DAST例外制御・並行安全・テスト強化：6施策） -> P2（セキュリティヘッダ・供給網・ドキュメント完成度向上：4施策）** の計12施策を定義する。



優先度フェーズ別 改善ロードマップ (P0 -> P1 -> P2)

フェーズ別実行計画とトレーサビリティ

- **Phase 1 (`P0` — 外部公開前の必須対応)**: `pom.xml` の `spring-boot-starter-parent` をパッチ適用済み系列（`>= 3.5.14` / `4.0.6`、または Tomcat `>= 10.1.58`・Jackson `>= 2.18.11`）へ更新して CRITICAL `8` 件・HIGH `29` 件を一括解消する。同時に `InventoryController`（57行目の `TODO(security)`、DAST `CWE-306` 実証）に対し `spring-boot-starter-oauth2-resource-server` による JWT スコープ認可と `sku` パターン検証を適用する。
- **Phase 2 (`P1` — 本番運用性・DAST 例外応答・スレッド安全性・テスト拡充)**: `@RestControllerAdvice` による RFC 7807 `ProblemDetail`（未知 SKU `IllegalArgumentException` の `404` 応答化による DAST `CWE-755` 解消、在庫不足の `409` 変換）と `spring-boot-starter-actuator` (`/actuator/health`) を追加し、`SupplierRestClient` の手動 JSON 連結を Jackson `ObjectMapper` へ、

`InventoryService.reserveStock` の `get` / `put` を `inventoryStore.compute(...)` へ置換する。併せて Ubuntu ベースイメージ刷新、`.pni/pentests/` 再検証、および MockMvc / HTTP サーバテスト追加により JaCoCo 行網羅率 `85%` 以上を達成する。

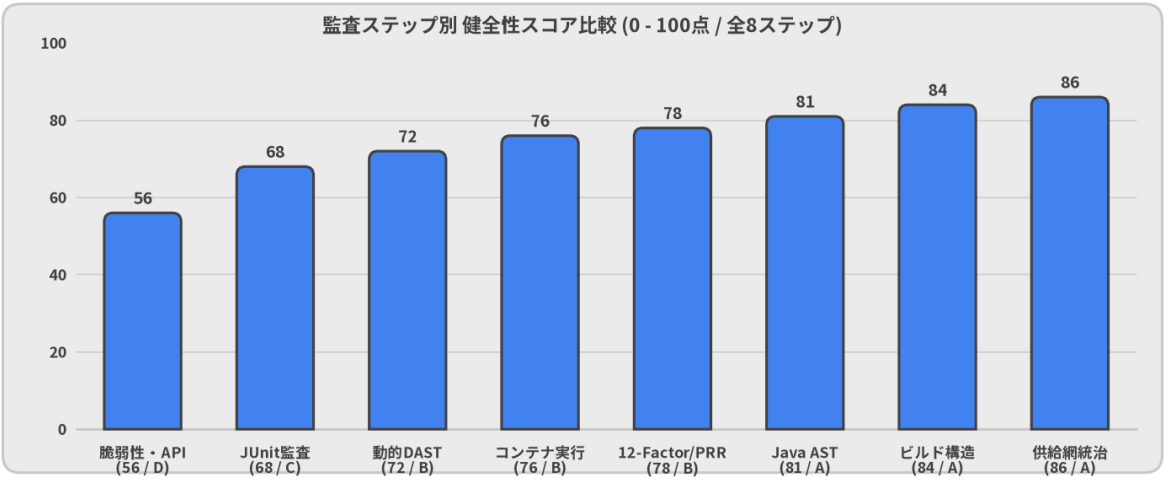
- **Phase 3 (`P2` – セキュリティヘッダ・サプライチェーン再現性とドキュメント整備)**: Spring Security 既定ヘッダによる `X-Content-Type-Options: nosniff` 付与（DAST `CWE-693` 解消）、`.mvn/wrapper/maven-wrapper.properties` を備えた完全版 `mvnw` 生成、`LICENSE` への Apache-2.0 全文記載、`Dockerfile` ベースイメージの `@sha256` ダイジェスト固定、および `README.md` の前提環境・設定一覧・`curl` 例追記を行う。

優先度	施策	対象ファイル	期待効果	関連ルール / CWE
P0	予約POST APIへのOAuth2 JWT認可とSKU形式検証導入	pom.xml .../api/InventoryController.java	無認証での在庫枯渇・外部通信誘発リスク(SEC-05)を遮断	common/security_vulnerability common/security
P0	Spring Boot BOM・Tomcat・Jackson のパッチ版更新	pom.xml	Maven推移依存のCRIT 8件・HIGH 29件を完全解消(SEC-01)	common/security_vulnerability CVE-2025-24813 他
P1	ProblemDetail例外変換・@Validated・Actuator導入	pom.xml, InventoryController, SupplierProperties.java	404/409標準応答・起動時設定検証/actuator/health プローブ確立	common/production_readiness java/frameworks/spring_boot
P1	Jackson JSON直列化とConcurrentHashMap競合解消	.../client/SupplierRestClient.java .../service/InventoryService.java	JSONインジェクション防止と同時引当時の在庫過販売防止	common/security java/code_quality
P1	未知SKU例外の404変換と.pni/pentests DAST全件合格化	.../api/InventoryController.java .../service/InventoryService.java	無認証401/403拒否と未知SKU HTTP 500解消をDASTで実証	common/security_vulnerability CWE-306 / CWE-755
P1	JREベースイメージ刷新とDS-0026 HEALTHCHECK追加	Dockerfile	Ubuntu OS層 HIGH 5件(OpenSSL, GnuPG)とDS-0026を解消	common/security_vulnerability CVE-2026-45447 / DS-0026
P1	JaCoCo行(62.7%)・分岐(50%)未実行パスのテスト網羅	pom.xml, SupplierRestClient, InventoryController, *Test.java	RV-03基準(行85% / 分岐75%)達成と jacoco:check 自動検証	common/runtime_verification java/testing
P1	MockMvcスライステスト・異常系assertThrows拡充	pom.xml, InventoryServiceTest, ReorderPolicyTest.java	GET 200/404・@Min(1)・例外境界の回帰防止とパッケージ階層整合	java/testing common/runtime_verification
P2	完全版 Maven Wrapper 生成とApache-2.0 LICENSE全文記載	mvnw, LICENSE, Dockerfile	SHA256検証付きMaven固定とOSSライセンス条項の完全明確化	common/supply_chain java/dependencies
P2	READMEへの前提条件・全設定プロパティ・curl例の追記	README.md	コード参照不要での迅速な導入とdefault-reorder-batch 設定把握	common/documentation
P2	X-Content-Type-Options: nosniff 保安ヘッダ出力設定	.../api/InventoryController.java pom.xml	MIMEスニッフィング抑止ヘッダ付与で test_baseline_http 合格	common/security_vulnerability CWE-693
P2	Dockerfile @sha256 固定とActuatorヘルスプローブ連携	Dockerfile pom.xml	ベースタグ書き換え耐性向上とオーケストレータ自動復旧連携	common/supply_chain common/runtime_verification

推奨改善アクション計画マトリクス (P0 -> P1 -> P2 全12施策)

4. 検査指標・ルール適合マトリクスと指摘一覧

本章では、`java_api_spring_clean` ワークフローで実行した全8監査ステップのスコア比較（弱点順）、評価対象となった全15エンジニアリングルールの適合判定マトリクス（`FAIL -> WARN -> PASS`）、および重要度順に正規化された全18件の指摘事項（`CRITICAL: 1` / `HIGH: 2` / `MEDIUM: 9` / `LOW: 6`）を網羅的に提示する。



実行監査ステップ別スコア比較 (弱点順: Weakest -> Strongest 全8ステップ)

ルール適合マトリクス（全15ルール：`FAIL: 1` / `WARN: 10` / `PASS: 4`）

全評価ルールは最悪ステータス優先で集約されており、Maven 推移依存の CRITICAL `8` 件・HIGH `29` 件および状態変更 `POST` ルートの認証欠如（ライブ DAST でも `CWE-306` / `CWE-755` / `CWE-693` を実証）により `common/security_vulnerability` の1件が `FAIL` 判定となった。一方、`common/architecture`・`common/git_workflow`・`common/maintainability`・`java/typing` の4ルールは違反ゼロで `PASS` を達成している。

判定	ルールID	代表評価ステップ	決定的エビデンス要約
FAIL	common/security_vulnerability	security_vulnerability_audit	Maven CVE 75件(CRIT 8/HIGH 29)・DAST CWE-306/755/693検出
WARN	common/documentation	repo_structure_audit	Diataxis 4章構成済だが前提環境・defaultReorderBatch・応答例が未記載
WARN	common/production_readiness	service_readiness_audit	HTTP timeout 設定済だが Actuator 不在・ドメイン例外が 500 応答
WARN	common/runtime_verification	container_runtime_audit	ビルド/テスト/起動合格も JaCoCo 実測行 62.7%・分岐 50.0%
WARN	common/security	java_ast_audit	秘密漏洩0件だが SupplierRestClient:45 で手動 JSON 文字列連結
WARN	common/supply_chain	repo_structure_audit	SHA固定CI・Dependabot済だが mvnw/LICENSE スタブ・DS-0026 LOW
WARN	common/twelve_factor	service_readiness_audit	@ConfigurationProperties 集約済だが ConcurrentHashMap インメモリ保持
WARN	java/code_quality	java_ast_audit	Javadoc/import 100% だが InventoryService:65 で get/put 非アトミック更新
WARN	java/dependencies	repo_structure_audit	Java 21・BOM固定済だが Actuator / spring-boot-starter-test / JaCoCo 不在
WARN	java/frameworks/spring_boot	java_ast_audit	コンストラクタDI徹底だが @Validated と ProblemDetail 例外変換が欠落
WARN	java/testing	java_test_audit	@Test 2件のみ。SupplierRestClient・GET /{sku}・例外境界が未テスト
PASS	common/architecture	java_ast_audit	循環依存 0件・境界違反 0件。api -> service -> domain/client/config
PASS	common/git_workflow	repo_structure_audit	.gitignore 整備済み(target/, .idea/)、main PR 自動テストゲート稼働
PASS	common/maintainability	java_ast_audit	全7ファイル 180 LOC(最大43 LOC)、複雑関数・過剰引数・大域状態 0件
PASS	java/typing	java_ast_audit	型安全率 100.0%、Optional<StockItem>・総称型徹底、raw型・キャスト0件

エンジニアリングルール適合マトリクス (FAIL -> WARN -> PASS 全15ルール)

重要度順 指摘事項一覧（全18件：`CRITICAL: 1` / `HIGH: 2` / `MEDIUM: 9` / `LOW: 6`）

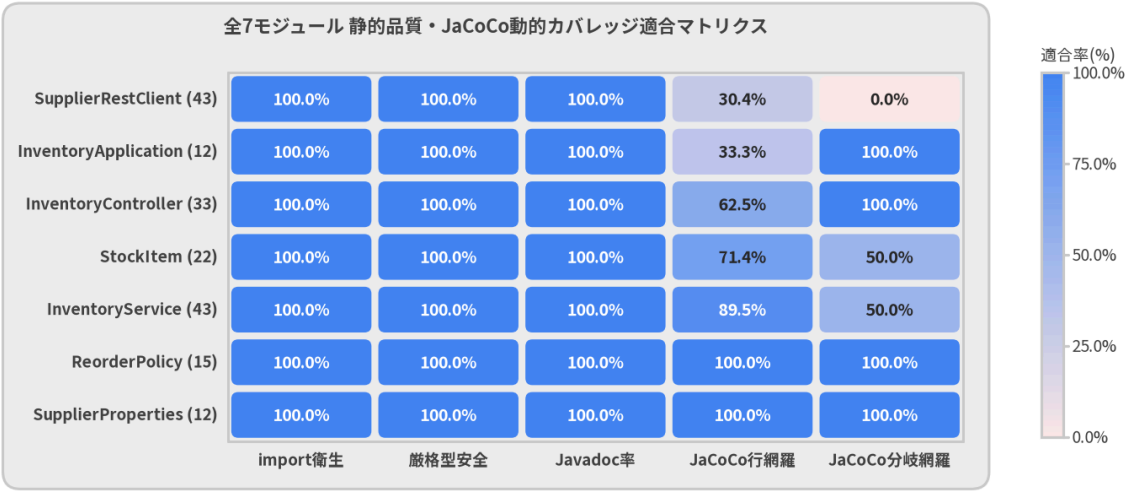
すべての指摘は実在するリポジトリ相対パス・行番号・決定的ファクト（`.pni/pentests/` ライブ HTTP 応答を含む）に紐付けられている。特に `pom.xml`、`InventoryController.java`、`SupplierRestClient.java`、`InventoryService.java`、`Dockerfile` の5ファイルに重要指摘が集約されており、変更影響を局所化したまま迅速に改修できる。

重要度	ルールID	該当箇所	検出事象・根拠	推奨修正策
CRITICAL	common/ security_vulnerability	pom.xml:10	Spring Boot 3.3.4 推移依存に CRIT 8件・HIGH 29件(計75件)	Spring Boot >=3.5.14 更新と Tomcat 10.1.58 / Jackson固定
HIGH	common/ security_vulnerability	.../api/ InventoryController.java:52	POST /{sku}/reserve が無認証 (57行目に TODO(security))	OAuth2 Resource Server 導入 ＋SKU @Pattern 形式検証
HIGH	java/testing	.../inventory/ InventoryServiceTest.java:27	Stub上書きで HTTP クライアント・ GET・例外パスのテストが0件	HttpServer・@WebMvcTest・ assertThrows テストを追加
MEDIUM	common/ production_readiness	.../api/ InventoryController.java:58	未知SKU・在庫不足の例外が 未捕捉で HTTP 500 応答化	@RestControllerAdvice で 404/409 ProblemDetail 変換
MEDIUM	java/frameworks/ spring_boot	.../config/ SupplierProperties.java:12	@ConfigurationProperties に @Validated・制約注釈が不在	@Validated と @NotBlank / @Positive で起動時検証
MEDIUM	java/code_quality	.../service/ InventoryService.java:65	ConcurrentHashMap への get/put 分離による同時引当競合リスク	inventoryStore.compute(...) で引当計算をアトミック化
MEDIUM	common/supply_chain	mvnw:3	exec mvn のみの4行スタブで Maven版・SHA256が未固定	mvn wrapper:wrapper で maven-wrapper.properties 生成
MEDIUM	common/security	.../client/ SupplierRestClient.java:45	文字列連結による手動 JSON 組立 で JSON 構造注入リスク	RestockRequest record を Jackson ObjectMapper 直列化
MEDIUM	common/ security_vulnerability	Dockerfile:7	jammy JRE に HIGH 5件(OpenSSL, GnuPG)＋DS-0026 HEALTHCHECK欠落	パッチ済 noble JRE 更新と HEALTHCHECK 命令の追加
MEDIUM	common/ security_vulnerability	.../api/ InventoryController.java:57	[DAST CWE-306] 無認証 POST が フィルタ不在で reserveStock 到達	Spring Security JWT 認可強制 (未認証時は 401/403 拒否)
MEDIUM	common/ security_vulnerability	.../service/ InventoryService.java:64	[DAST CWE-755] 未知SKU引当で IllegalArgumentException -> 500	ExceptionHandler で未知SKUを 404 ProblemDetail へ変換
MEDIUM	common/ runtime_verification	.../client/ SupplierRestClient.java:44	JaCoCo 行 62.7%・分岐 50.0% (Client 30.4% / API 62.5%)	未実行行(Client 45-55行等)を 網羅し jacoco:check 設定
LOW	common/twelve_factor	.../service/ InventoryService.java:23	インメモリ ConcurrentHashMap 直保持で複数レプリカ共有不可	InventoryRepository 抽出と server.shutdown=graceful 設定
LOW	common/supply_chain	LICENSE:1	Apache-2.0 見出し3行のみで 許諾条項・免責全文が欠落	SPDX 標準の Apache-2.0 ライセンス全文を記載
LOW	common/documentation	README.md:14	JDK 21/Docker 前提・一部設定・ curl リクエスト/応答例が未記載	前提環境・全3設定項目・ HTTP ステータス例を追記
LOW	common/ security_vulnerability	.../api/ InventoryController.java:18	[DAST CWE-693] 応答ヘッダに X-Content-Type-Options 不在	nosniff および no-store セキュリティヘッダを付与
LOW	common/supply_chain	Dockerfile:7	ベースイメージがタグ指定のみで @sha256 ダイジェスト未固定	builder / JRE 双方のイメージを @sha256 ダイジェストで固定
LOW	java/testing	.../inventory/ ReorderPolicyTest.java:1	テスト2クラスがルート直下に置かれ api/service/domain と不一致	src/test/java 配下のパッケージを src/main/java 階層に整合

重要度順 検出指摘一覧 (CRITICAL -> HIGH -> MEDIUM -> LOW 全18件)

5. 詳細分析：コード品質・テスト・動的検証

本章では、Java AST 静的解析によって計測されたモジュール単位のコード品質・SIG 保守性・ジェネリクス型安全性指標と、隔離 Docker コンテナ（Java 21 / Maven 3.9.9）上で実行されたマルチステージ `Dockerfile` ビルド、オフライン JUnit 5 単体テスト、JaCoCo 実測カバレッジ、および Spring Boot 起動スモーク検証の結果を詳細に分析する。



モジュール × 品質ディメンション適合度マトリクス (0 - 100%)

静的コード品質・SIG 保守性・並行処理安全性の評価

- **SIG 保守性・型安全・Javadoc 衛生**: 全7ソースファイル合計 `180` LOC（最大は `InventoryService.java` と `SupplierRestClient.java` の各 `43` LOC）と極めて凝集度が高く、複雑関数（`0` 件）・過剰引数（`0` 件）・ワイルドカード import（`0` 件）・raw 型（`0` 件）・Javadoc 欠落（`0` 件）すべてで `100.0%` 適合を達成している。
- **並行リクエスト時の競合状態（Check-Then-Act）**: `InventoryService.reserveStock`（65〜70行目）では `inventoryStore.get(sku)` で取得した `StockItem` に対して `withAdditionalReservation` を計算した後に `inventoryStore.put(sku, updated)` を呼び出している。同一 SKU へ同時リクエストが到達した際、2スレッドが同じ `existing`

を読み取って片方の引当を上書き消失させる競合窓が存在するため、`inventoryStore.compute(...)` によるアトミック遷移への改修が必要である。

- **Spring 設定検証・例外ハンドリング**: `SupplierProperties` は `@ConfigurationProperties(prefix = "inventory.supplier")` の不変 `record` だが `@Validated` が未付与であり、また `InventoryController` には未知 SKU（`IllegalArgumentException`）や在庫不足（`IllegalStateException`）を HTTP `404` / `409` へ写像する `@RestControllerAdvice` が未実装である。

コンテナ動的検証（マルチステージビルド・オフライン JUnit 5・JaCoCo・起動検証 & DAST 連携）

PNI の `run_check` により、リポジトリ同梱マルチステージ `Dockerfile` のクリーンビルド（`44.38s`）および Java 21 ハーネスビルド（`1.03s`、キャッシュ済レイヤー活用）を実行し、ネットワーク完全遮断（`-o` かつ `--network=none`）環境下で JUnit 5 単体テスト（`2 / 2` 件合格、`11.04s`）と JaCoCo `0.8.12` カバレッジ計測を実施した。さらに `InventoryApplication` の起動スモークチェックでは `4.51s` でポート `18080`（`tcp://127.0.0.1:18080`）への正常バインドが確認された。加えて、後続の `web_pentest`（`.pni/pentests/examples/java/api_spring_clean/`、`7.19s`）によるライブ HTTP 検証でも不正型パラメータ `quantity=not-an-int` の `400 Bad Request` 拒否が動作確認された一方、単体テスト未網羅であった `InventoryService.java:67` の未知 SKU 例外パスがライブ HTTP 上で未処理 `500`（`CWE-755`）を返すことが実証された（詳細は第6章参照）。

PASSED (44.4s)

マルチステージ Dockerfile

ハーネスビルドも合格 (1.03s)

2 / 2 PASS

オフライン JUnit 5

--network=none / -o (11.0s)

62.7% / 50.0%

JaCoCo 行 / 分岐網羅率

42 / 67 命令列 (目標85%/75%)

BOUND (4.51s)

Spring 起動 & DAST (12/16)

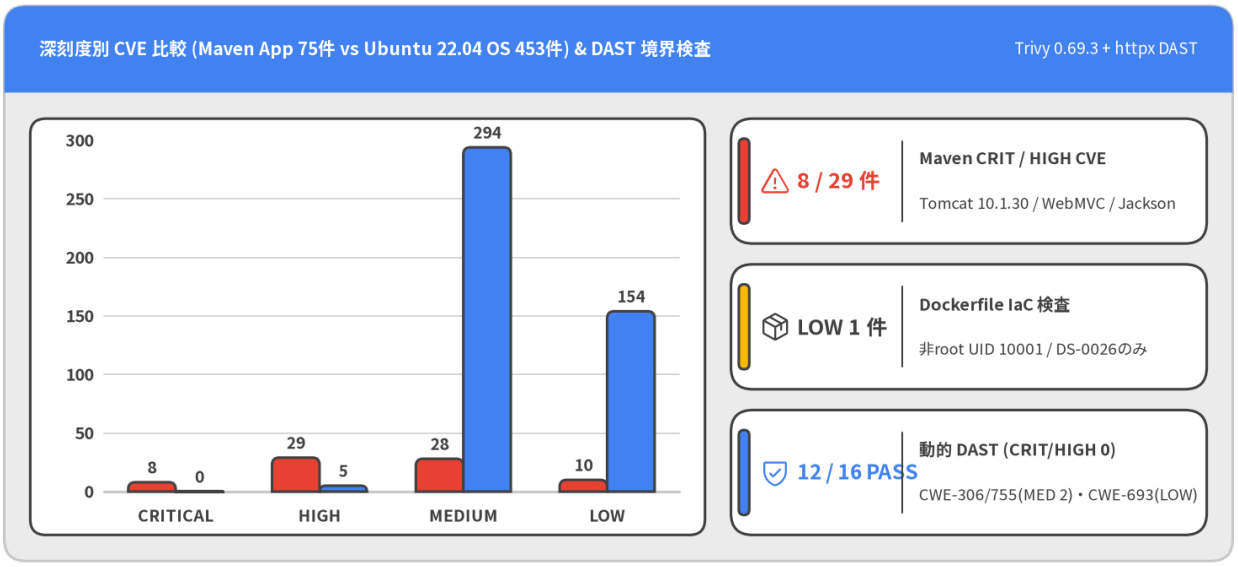
127.0.0.1起動合格・Actuator無

対象モジュール (行網羅率昇順)	行網羅率	分岐網羅率	実行 / 総行数	未実行行 (missing_lines_preview)
client/SupplierRestClient.java	30.4%	0.0%	7 / 23	45-55行 (requestRestock 通信・例外処理全体)
InventoryApplication.java	33.3%	N/A	1 / 3	21, 22行 (SpringApplication.run エントリ)
api/InventoryController.java	62.5%	N/A	5 / 8	40, 41, 42行 (getItem 200/404 応答パス)
domain/StockItem.java	71.4%	50.0%	5 / 7	40, 43行 (数量<=0・在庫不足の例外送出分岐)
service/InventoryService.java	89.5%	50.0%	17 / 19	54行 (findBySku), 67行 (未知SKU例外分岐)
domain/ReorderPolicy.java	100.0%	100.0%	5 / 5	なし (発注点判定・補充量計算の全分岐網羅)
config/SupplierProperties.java	100.0%	N/A	2 / 2	なし (record コンストラクタ・defaults 網羅)

コンテナ動的検証KPIと全7モジュール JaCoCo 実測カバレッジ内訳

6. 詳細分析：セキュリティ・脆弱性・API 境界

本章では、Trivy (`0.69.3`) による `pom.xml` および解決済みマルチステージ Docker イメージ（`eclipse-temurin:21.0.4_7-jre-jammy`）に対する SCA 脆弱性スキャン・Dockerfile IaC 設定検査・秘密情報スキャン、AST による危険な CWE コードシンク・Spring Boot HTTP API 攻撃面の評価、ならびに `.pni/pentests/examples/java/api_spring_clean/` を用いた隔離ループバック動的 Web ペネトレーションテスト（`web_pentest` / `SEC-06`）の結果を統合して分析する。



Maven アプリケーション依存 vs Ubuntu OS層 CVE内訳と Dockerfile / DAST 検査指標

アプリケーション CVE vs OS 層 CVE・IaC・API 境界評価（SEC-01〜SEC-05）

- **Maven アプリケーション依存関係 (`SEC-01` ー `FAIL`)**: `pom.xml` の `spring-boot-starter-parent:3.3.4` から解決される推移依存に計 `75` **件**（`CRITICAL: 8`, `HIGH: 29`, `MEDIUM: 28`, `LOW: 10`）の CVE が検出された。特に組み込み Tomcat (`tomcat-embed-core@10.1.30`) の Partial PUT RCE（`CVE-2025-24813`, CVSS 10.0）や認証バイパス群

（`10.1.58` で修正）、`spring-webmvc@6.1.13` の `XsltView` RCE（`CVE-2026-47884`, CVSS 9.8）、および `jackson-databind` / `jackson-core@2.17.2` の任意コード実行・DoS 群（`2.18.11` で修正）は最優先で BOM アップグレードが必要である。

- **コンテナ OS 層と Dockerfile ハードニング (`SEC-01` / `SEC-02`)**: `eclipse-temurin:21.0.4_7-jre-jammy`（Ubuntu 22.04）OS 層では `CRITICAL` は `0` 件だが、`libssl3` / `openssl`（`CVE-2026-45447`, CVSS 8.8）と `gpgv`（`CVE-2025-68973`, CVSS 7.8）の `HIGH` 計 `5` 件が検出された。Dockerfile 自体は非 root ユーザ `spring`（`UID 10001`）と `-Dserver.address=127.0.0.1` を徹底しており、IaC 指摘は `HEALTHCHECK` 未定義（`DS-0026`, `LOW`）1件のみである。
- **秘密情報・CWE コードシンク・Spring API 攻撃面 (`SEC-03` 〜 `SEC-05`)**: 秘密情報漏洩（`0` 件）および危険な CWE コードシンク（SQLi / OS コマンド注入 / 安全でないデシリアライズ等 `0` 件）は完全にクリーンである。一方、公開された 2 つの REST エンドポイントはいずれも認証ヒントを持たず、特に状態変更と外部 HTTP 発信を伴う `POST /api/v1/items/{sku}/reserve` には `TODO(security)` コメントが残存しており、外部 Ingress 公開前に OAuth2 JWT 認可と `SupplierRestClient` の Jackson シリアライズ化が必須である。

区分 / 深刻度	ルート / CVE・検査ID	ハンドラ・対象パッケージ	現状 -> 推奨修正・更新先	リスク・境界検査の詳細
API [HIGH]	POST /api/v1/items/{sku}/reserve	InventoryController.reserveItem (:52)	auth=NONE (@Min(1)のみ) -> OAuth2 JWT + @Pattern	無認証で在庫引当と外部補充POSTを誘発可能 (57行目 TODO security)
API [INFO]	GET /api/v1/items/{sku}	InventoryController.getItem (:38)	auth=NONE (読取専用) -> 公開範囲に応じJWT保護	SKU指定の単一在庫参照。未知SKU時はOptional空から 404 を返却
App [CRITICAL]	CVE-2025-24813 (他 CRIT 6件)	tomcat-embed-core (pom.xml)	10.1.30 -> 10.1.58 (または 11.0.25)	Partial PUT RCE (CVSS 10.0)・HTTP/2未検証・認証バイパス
App [CRITICAL]	CVE-2026-47884 (+HIGH 5件)	spring-webmvc / core / expression (pom.xml)	6.1.13 -> 6.2.19 / Spring Framework 7.0.9	XsltView パス制限不備による RCE (CVSS 9.8)・SpEL/静的資源 DoS
App [HIGH]	CVE-2026-54512 CVE-2026-54513 他6件	jackson-databind / jackson-core (pom.xml)	2.17.2 -> 2.18.11 (または 2.21.7)	PolymorphicTypeValidator 回避による任意コード実行・JSON DoS
App [HIGH]	CVE-2025-22235 CVE-2026-40973	org.springframework.boot: spring-boot (pom.xml)	3.3.4 -> 3.5.14 (または 4.0.6)	Actuator マッチャ不整合および予測可能一時ディレクトリ悪用
OS [HIGH]	CVE-2026-45447 (計2パッケージ)	libssl3 / openssl (ubuntu 22.04)	3.0.2-0ubuntu1.18 -> 3.0.2-0ubuntu1.25	OpenSSL PKCS7_verify() における Heap Use-After-Free (CVSS 8.8)
OS [HIGH]	CVE-2025-68973	gpgv (ubuntu 22.04)	2.2.27-3ubuntu2.1 -> 2.2.27-3ubuntu2.5	GnuPG 境界外書込による情報漏洩・任意コード実行リスク (CVSS 7.8)
IaC [LOW]	DS-0026 (SEC-02)	Dockerfile (eclipse-temurin JRE)	HEALTHCHECK 未定義 -> /actuator/health 監視	非root(UID 10001)・127.0.0.1固定済 ヘルスチェック命令のみ未設定
SAST / Secret	SEC-03 / SEC-04	全Javaソース・設定群	漏洩 0件 / CWEシンク 0件 -> Jackson直列化で完全化	秘密情報0・CWE-78/89/502 0件 SupplierRestClient:45のみ文字列連結

Spring Boot API攻撃面マトリクスおよび主要CVE・Dockerfile検査詳細一覧

動的Webペネトレーションテスト (`web_pentest` / `.pni/pentests/` ー SEC-06)

PNI の `web_penetration_audit` ステップ（スコア `72 / 100`, Grade `B`, ルール判定 `WARN`）では、`.pni/pentests/examples/java/api_spring_clean/` に実体化された4ファイル構成の Python + `pytest` + `httpx` ペネトレーションスイート（`conftest.py`, `test_app_probes.py`, `test_baseline_http.py`, `test_target_surface.py`）を用い、隔離ループバックコンテナ（`http://127.0.0.1:53369`, `server_booted=true`）に対して計 `16` **プローブ**をライブ実行した（所要 `7.19s`：`12` PASS / `3` FAIL / `1` SKIP / `0` ERROR、CRITICAL・HIGH 脆弱性 `0` 件）。

- **ライブ実証された防御境界 (`12` PASS)**: `POST /api/v1/items/SKU-100/reserve?quantity=not-an-int` が `HTTP 400` で安全に型拒否されること（`CWE-20` PASS）に加

え、`GET /api/v1/items/1` に対する反射型 XSS カナリア（`CWE-79`）、SQL エラー・スタックトレース露出（`CWE-89` / `CWE-209`）、`../../../../etc/passwd` パストラバーサル（`CWE-22`）、ならびに `.env` / `.git/HEAD` / ワイルドカード CORS 検査（`CWE-538` / `CWE-942`）がすべて合格した。

- **ライブ検出された3件のエンドポイント・ヘッダ課題 (`2` MEDIUM / `1` LOW)**:
  1. `[MEDIUM] CWE-306` (`test_app_probes.py` -> `InventoryController.java:57`): 無認証の `POST /api/v1/items/SKU-UNAUTH-PROBE/reserve?quantity=1` が認証フィルタに遮断されず `InventoryService.reserveStock()` まで到達し、`HTTP 500` を返却した（期待値：`401` / `403`）。
  2. `[MEDIUM] CWE-755` (`test_app_probes.py` -> `InventoryService.java:64`): 未知 SKU に対する `POST /api/v1/items/SKU-MISSING-999/reserve?quantity=1` で `IllegalArgumentException("Unknown SKU: ...")` が未捕捉のまま送出され、制御された `404 ProblemDetail` ではなく未処理の `HTTP 500` となった。
  3. `[LOW] CWE-693` (`test_baseline_http.py` -> `InventoryController.java:18`): `GET /api/health -> 404` 応答に `X-Content-Type-Options: nosniff` ヘッダが付与されていない。



結果 / 深刻度	CWE / プローブファイル	対象エンドポイント・コード箇所	ライブHTTP観測エビデンスと改善アクション
FAIL [MEDIUM]	CWE-306 test_app_probes.py	POST /api/v1/items/{sku}/reserve無認証 InventoryController.java:57	POST ?quantity=1 が認証フィルタ不在で reserveStock へ到達し HTTP 5(-> Spring Security OAuth2 JWT 認可で 401/403 拒否を強制
FAIL [MEDIUM]	CWE-755 test_app_probes.py	POST /api/v1/items/{sku}/reserve InventoryService.java:64	POST /SKU-MISSING-999/reserve?quantity=1 で未知SKU例外が未処理 HTTP 5(-> @RestControllerAdvice で IllegalArgumentException を 404 ProblemDetail (
FAIL [LOW]	CWE-693 test_baseline_http.py	GET /api/health (全応答共通) InventoryController.java:18	GET /api/health -> 404 応答で X-Content-Type-Options ヘッダが欠落 -> Spring Security 既定ヘッダで X-Content-Type-Options: nosniff を常時付与
PASS [検証済]	CWE-20 test_app_probes.py	POST /api/v1/items/SKU-100/reserve (@RequestParam int quantity)	POST ?quantity=not-an-int -> HTTP 400 Bad Request を返却し 500 化を防止
PASS [検証済]	CWE-79 / 89 / 209 / 22 test_target_surface.py	GET /api/v1/items/1 & ベースライン露出検査	XSSカナリア反射なし・SQL/スタックトレース漏洩なし・../etc/passwd 漏洩なし (.env / .git / 危険CORS 露出も test_baseline_http.py 9プローブで PASS)

動的Webペネトレーションテスト (.pni/pentests/) 実行サマリーとプローブ別検証結果